# Examination Demonstration & Viva Guide

**Course Code**: 20CYS401 — Secure Software Engineering
**System**: Online Book Review and Recommendation Platform
**Target Audience**: Examination Evaluator / Viva Examiner

## 1. Quick Demonstration Workflow (3-5 Minutes)

### Step 1: Start the Application

```
bash
cd /Users/abhinaydasari/Documents/sse/secure-book-review-platform
./venv/bin/python wsgi.py
```

Open browser at `http://localhost:5000`.

### Step 2: Demonstrate Normal User Workflow

1. Log in as `alice` (Password: `AlicePass123!`).
2. Search for a book (e.g. "Security" or "Python") or filter by genre.
3. Open a book details page (e.g. *Threat Modeling: Designing for Security*).
4. Add the book to the personal reading list shelf (`Currently Reading`).
5. Submit a rating (5 stars) and a review.
6. Verify the review appears with an **Edit** and **Delete** option visible to Alice.
7. Navigate to `/my-shelf` and demonstrate the private shelf view.
8. Log out.

### Step 3: Demonstrate Security Controls & RBAC

1. Log in as `bob` (Password: `BobPass123!`).
2. Navigate to the same book. Observe that Alice's review does NOT have an Edit or Delete button for Bob.
3. Attempt to access `/admin/books` directly as Bob → System blocks Bob with **HTTP 403 Forbidden** (guarded by `@admin_required`).

4. Log out.

### Step 4: Demonstrate Administrator & Moderation Workflow

1. Log in as `admin` (Password: `AdminPass123!`).
2. Navigate to `/admin/books` → Add a new book or edit an existing catalog record.
3. Navigate to `/admin/moderation` → View the community reviews queue.
4. Moderate/hide an inappropriate review by providing a mandatory moderation reason.
5. Review the real-time **Audit Log** table at the bottom of the page showing tamper-evident logs for recent logins, review creations, and moderation actions.
6. Log out.

## 2. Technical Security Demonstrations (Examiner Questions)

### Question 1: "How do you prove your system defends against IDOR / BOLA?"

**Answer & Demo**:

1. Show the code in `app/services/review_service.py` (`update_review`):

   ```
   python
   if not review.is_owned_by(user_id):
       return False, "Access denied: You can only edit your own reviews."
   ```

2. Demonstrate the automated test proof:

   ```
   bash
   ./venv/bin/pytest tests/test_authorization.py -v
   ```

   Both `test_user_b_cannot_edit_user_a_review` and `test_user_b_cannot_delete_user_a_review` pass.

3. Reference the dedicated demonstration branch `demo/idor-before` where omitting this check allowed Bob to overwrite Alice's review (evidence in `evidence/phase-12/idor_vulnerable_proof.txt`).

### Question 2: "How is SQL Injection prevented?"

**Answer & Demo**:

- 100% of data queries use SQLAlchemy ORM ( `app/services/search_service.py` ).
- Run the SQLi test:

  ```
  bash
  ./venv/bin/pytest
  tests/test_unit_search.py::test_search_sql_injection_metacharacters -v
  ```

  Special metacharacters ( `' OR 1=1 --` , `UNION SELECT` ) are safely treated as literal search strings, returning 0 matches without error.

### Question 3: "How is XSS prevented?"

**Answer & Demo**:

- Jinja2 auto-escaping is active across all templates in `app/templates/` .
- Run the XSS test:

  ```
  bash
  ./venv/bin/pytest tests/test_fuzzing.py::test_fuzz_xss_payloads_in_review -v
  ```

  Payloads like `<script>alert('XSS')</script>` are rendered strictly as sanitized, escaped text strings.

### Question 4: "Where are security scans and CI/CD defined?"

**Answer & Demo**:

- Static analysis: Bandit SAST scan report saved at `evidence/phase-11/bandit_scan.txt` (0 High, 0 Medium).
- CI/CD: Pipeline defined in `.github/workflows/ci.yml` running Pytest, Bandit, and container policy checks on every push.
- Containerization: `docker/Dockerfile` uses multi-stage builds and runs under unprivileged UID 10001 with active healthcheck probes.